CPS188 : Lab 02 - Programming Techniques
Name: Iqraa Wahid
Student Number: 501287410

Problem #1:

Program	Error Message (text please - no screenshots) (copy and pasted from Geany, compiler)	Cause of Error	Correction Applied
program01.c	DOUBLE x, y, result; error: unknown type name 'DOUBLE'	The type double is represented by 'double' not 'DOUBLE' as represented in the code.	`double x, y, result;`
program02.c	scanf ("%lf", y); warning: format '%lf' expects argument of type 'double *', but argument 2 has type 'double' [-Wformat=]	The “&” is missing in the following statement: scanf ("%lf", y) When using the function scanf for the above statement, the “%lf” acts as a placeholder for a type double variable, whereas the “&y” allows for the software to know where exactly the input value will be stored.	`scanf ("%lf", &y);`
program03.c	return (aver); error: incompatible types when returning type 'double (*)(double, double)' but 'double' was expected	The *aver* function should be returning the variable “ave” when called in the main function, this variable was initialized in the function *aver.* However, in the function, *aver,* the return statement was returning the function itself: return (aver);	`return (ave);`
program04.c	printf ("The average is %lf.\n", aver); warning: format '%lf' expects argument of type 'double', but argument 2 has type 'double (*)(double, double)' [-Wformat=]	When outputting the last statement in the main function, the placeholder stored the function *aver* itself: printf ("The average is %lf.\n", aver); when we were supposed	`printf ("The average is %lf.\n", result);`

		to, rather, output the value returned from the function *aver.* In order to do that, we were supposed to store the variable “result” in the placeholder outputting the last statement, like: printf ("The average is %lf.\n", result); This is because the type double variable “result” calls on the function *aver,* and stores the value returned from it.	
program05.c	result = aver (&x,y); error: incompatible type for argument 1 of 'aver'	When calling the function *aver* in the main function, we cannot call it in the following way: result = aver (&x,y); This is because the *aver* function takes in two arguments, by adding a “&” we are not matching the requirements of the *aver* function, since the *aver* function requires two arguments that are values.	`result = aver (x,y);`
program06.c	No error message.	A calculation error is the cause of the error. We are trying to calculate the average of two values, in order to do that we would have to add the values, and divide it by the number of values there are, two. However, the following code doesn’t follow the order of operation properly: ave = n1 + n2 / 2.0; The above code would first divide n2 by 2.0, then add n1 to the quotient, which is the wrong order of operation.	`ave = (n1 + n2) / 2.0;`

program07.c	int aver (double n1, double n2) error: expected declaration specifiers before 'ave'	A curly bracket was missing after initializing the function called *aver.* Curly brackets are supposed to enclose the body of a function.	double aver (double n1, double n2) {
program08.c	return (result); error: 'result' undeclared (first use in this function)	The function *aver* was returning a variable that was not even defined in the function, *aver,* itself. Furthermore, the aver should be a double type function, since the variable the function is returning is double.	`return (ave);`
program09.c	result = aver (x,y) error: expected ';' before 'printf'	There was a semicolon missing at the end of a statement: result = aver (x,y)	`result = aver (x,y);`
program10.c	printf "The average is %lf.\n", result); error: expected statement before ')' token	There was a missing opening bracket. When printing out a statement, using the printf function, whatever is wanted to be printed out must be enclosed with round brackets.	`printf ("The average is %lf.\n", result);`
program11.c	double aver (double x, double y) { error: 'n1' undeclared (first use in this function) error: 'n2' undeclared (first use in this function)	The variables n1 and n2 in the function, *aver*, were not declared in the argument of the function *aver*.	`double aver (double n1, double n2){`
program12.c	scanf ("%2.2lf", &x); scanf ("%2.2lf", &y); warning: unknown conversion type character '.' in format [-Wformat=]	In the scanf function, precision cannot be done.	`scanf ("%lf", &x);` `scanf ("%lf", &y);`
program13.c	result = aver (x); error: too few arguments to function 'aver'	When the function *aver* was called in the main function, it was called with only one parameter being assigned a value, when	`result = aver (x,y);`

		the function had required two arguments.	

Problem #2:

Algorithm:

1. Ask the user for the temperature in degrees celsius
2. Convert the temperature from degrees celsius to degrees fahrenheit
3. Use the temperature, in degrees fahrenheit, to calculate the speed of the sound, in feet per second, using the following: $sound\ =\ 1086\sqrt{(5(temperature)\ +\ 297)\ /\ 247}$
4. Convert the speed of the sound from feet per second to kilometers per hour
5. Display the speed of the sound in kilometers per hour.

Code:

```
#include <stdio.h>
#include <math.h>

double spsound (double t){
           double speedfps, tF, speedkph;//declaring variables used in this
function
           // converting the temperature from degrees celsius to degrees
fahrenheit
           tF=t*(9.0/5.0)+32.0;
           // calculate the speed in feet per second, using the temperature
in degrees fahrenheit
           speedfps = 1086.0 * sqrt(((5.0*tF)+297.0)/247.0);
           // convert the speed of sound from feet per second to kilometers
per hour
           speedkph = speedfps*1.09728;
           //return the speed in kilometers per hour
           return speedkph;
     }//closing function spsound

int main (void){
     double t, speed;//declare the variables used in this function

     printf("Enter the temperature (in degrees celsius): ");//ask the user to
enter the temperature in degrees celsius
```

```
	scanf("%lf", &t);//stores the user's response in the variable t

	speed = spsound(t);//call the function spsound, with argument with value of temperature user entered
	printf("The speed of the sound is %lf kilometers per hour at %lf degrees celsius.", speed, t);//output the results
	return (0);//closing statement
}//closing main function
```

Output:

```
Enter the temperature (in degrees celsius): 10
The speed of the sound is 1773.341899 km/h in 10.000000 degrees celsius.
```

Screenshot:

```
Enter the temperature (in degrees celsius): 10
The speed of the sound is 1773.341899 km/h in 10.000000 degrees
    celsius.

=== Code Execution Successful ===
```